تقرير الفحص العام لنظام Cashbook

الواجهة الخلفية (ASP.NET Core 8 + SQL Server) · تاريخ الفحص: 29 أكتوبر 2026 · الفرع: `master`

شمل الفحص المشروع كاملاً (نحو 10 آلاف سطر): المتحكمات والمستودعات والمصادقة والوسائط (Middleware) وسجل التدقيق وإعدادات التشغيل. **لم يُعدَّل أي ملف أثناء الفحص.** النقاط مرتبة حسب الخطورة، وفي آخر التقرير اقتراح لترتيب الإصلاح.

3	8	8	7	10
حرجة	عالية	أداء	أمنية متوسطة	جودة الكود

> يظهر أن جزءاً كبيراً من الكود مستخرج بأداة فك تجميع (Decompiler)، فهو يحتوي على `if (1 == 0) {}` و `goto IL_0062` و `IL_0777: throw null;` و `Exception ex2 = ex;` . هذا يفسّر صعوبة قراءته، وتفصيله في قسم جودة الكود.

أولاً: مشاكل حرجة (أمنية، يجب إصلاحها قبل أي شيء)

1. التصنيفات وجهات الاتصال وطرق الدفع بلا أي فحص صلاحية

الموقع: `PaymentMethodController.cs` · `ContactController.cs` · `CategoryController.cs`

المشكلة: يكتفي الكود بفحص أن المستخدم مسجّل دخوله، ولا يتحقق من عضويته في المنشأة ولا من دوره.

الأثر: أي مستخدم مسجّل، حتى لو أنشأ حسابه للتو عبر `register` ، يستطيع عرض وإضافة وتعديل وحذف تصنيفات أي منشأة وجهات اتصالها (أسماء وأرقام هواتف) إذا عرف معرّفها.

الحل: دالة فحص موحّدة مثل `CheckWalletAsync` تستخرج المنشأة من السجل نفسه، لا من الطلب، وتتحقق من الدور قبل كل عملية.

2. ثغرة قراءة حركات وسجلات خزائن منشآت أخرى (IDOR)

الموقع: `:96` · `TransactionHistoryController.cs:53` · `:160` · `TransactionController.cs:111`

المشكلة: يُفحص دور المستخدم في `businessId` القادم من الطلب، ثم تُقرأ بيانات `bookId` أو `transactionId` دون التأكد أنها تابعة لتلك المنشأة.

الأثر: عضو في منشأة A يرسل معرّف منشأته مع `bookId` من منشأة B فيقرأ كل حركاتها وأرصدتها وسجلها. وأيضاً تتجاوز الأدوار المقيّدة بخزائن محددة (مثل staff وdataoperator) قيدها في القراءة.

السبب: الإصلاح الأخير «عزل الخزائن بين المنشآت» طُبّق على عمليات الكتابة فقط.

الحل: استخدام `CheckWalletAsync(userId, bookId, Roles.TransactionRead)` نفسه في مسارات القراءة الأربعة.

3. رمز التحديث (Refresh Token) يعمل كرمز دخول، والعكس صحيح

الموقع: `UserRepository.cs:228` · `:302`

المشكلة (أ): يحمل رمز التحديث نفس claims رمز الدخول وموقّع بنفس المفتاح، لذلك يقبله النظام كرمز دخول صالح لمدة **20 يوماً** بدل 7 ساعات.

المشكلة (ب): يُخزَّن رمز التحديث بخوارزمية BCrypt، وهي لا تقرأ إلا أول 72 بايت. وأول 72 بايت في كل رموز JWT لهذا النظام ثابتة تقريباً (الترويسة وبداية اسم الـ claim)، فينجح فحص التحقق غالباً مع أي رمز لنفس الجلسة، بما في ذلك رمز الدخول نفسه ورموز التحديث القديمة. (استنتاج من قراءة الكود، يُثبت باختبار بسيط قبل الإصلاح.)

الأثر: من يسرق رمز دخول قصير العمر يستطيع تجديده إلى ما لا نهاية، ولا فائدة فعلية من تدوير رموز التحديث.

الحل: إضافة claim باسم `token_type` يُرفض في الوسيط إن كان refresh، وتخزين رمز التحديث بـ SHA-256 بدل BCrypt (أو جعله نصاً عشوائياً لا JWT).

ثانياً: مشاكل عالية (سلامة البيانات ومنطق العمل)

#	المشكلة	الموقع	الأثر	الحل
4	إنشاء الحركة يتم على 3 إلى 4 مراحل `SaveChanges` منفصلة بلا Transaction في قاعدة البيانات	`CreateTransactionAsync`	لو فشل رفع المرفق أو كتابة السجل تبقى حركة ناقصة بلا سجل أو بلا حقولها المخصصة	جمع الكل في `SaveChanges` واحد، أو `BeginTransactionAsync`
5	تعديل الحركة يحذف المرفقات القديمة من القرص **قبل** الحفظ	`UpdateTransactionAsync`	لو فشل الحفظ ضاعت المرفقات نهائياً	الحذف من القرص بعد نجاح الحفظ فقط
6	سجل الحركات ينسب التعديل والحذف إلى **منشئ الحركة** لا إلى من نفّذهما	`TransactionRepository.cs:146` و `:696`	السجل يقول إن فلاناً عدّل الحركة وهو لم يفعل، وهذا خطير في نظام مالي	تمرير `callerId` للدالتين
7	نسخ الحركة إلى خزنة أخرى لا ينسخ `ExchangeRate` و `ExchangeDate` ولا يكتب سجلاً	`DuplicateTransactionToAnotherBookAsync`	تنتج حركة دولار بلا سعر صرف، فتختل حسابات التحويل	نسخ الحقلين وإضافة سجل POST
8	`GetAsync` و `GetAllAsync` يعدّلان أي نص يبدأ بـ `{` أو `[` عبر Reflection	`Repository.cs:52-105`	في `GetAsync` يُعاد تنسيق النص وقد يُحفظ معدّلاً لأن الكيان متتبَّع. وفي `GetAllAsync` يُسند كائن JArray إلى خاصية نصية فينهار الطلب بخطأ 500	حذف هذا الكود كاملاً، فلا داعي له
9	إنشاء خزنة: المستودع يبتلع الاستثناء ويعيد false، والمتحكم يتجاهل النتيجة ويرد «تم الإنشاء»	`BookController.cs:160`	يظهر للمستخدم نجاح والخزنة لم تُنشأ. وفحص `null` يأتي بعد استخدام الكائن (سطر 148 قبل 156)	فحص القيمة المرجعة، وتقديم فحص null

10	في تعديل التصنيف يُنفَّذ `Map` قبل فحص `null`	`CategoryControll er.cs:191`	خطأ 500 بدل 404. ويتكرر النمط في Contact وPaymentMethod	ترتيب الفحص قبل الاستخدام
11	كثير من كتل `catch` تعيد `_response` بحالة HTTP 200 مع `IsSuccess=false`	Book وCategory وContact وغيرها	الواجهة الأمامية تعامل الخطأ كنجاح	توحيد معالجة الأخطاء (البند 22)

ثالثاً: مشاكل الأداء

#	المشكلة	الأثر	الحل
12	**قائمة الحركات المقسّمة إلى صفحات** تحسب الرصيد التراكمي (`SUM OVER`) على **كل حركات الخزنة** في كل طلب صفحة، ومعه `COUNT` بأربعة JOIN حتى لو لم يُطلب أي فلتر	يزداد بطء الطلب مع عدد الحركات. خزنة بـ100 ألف حركة تُمسح كاملة مع كل صفحة	(أ) فهرس مغطٍّ `(BookId, Date, Id) INCLUDE (Amount, Currency, Type)`. (ب) JOIN في استعلام العدد فقط عند وجود فلتر. (ج) على المدى البعيد: رصيد افتتاحي شهري أو عمود رصيد محسوب
13	لا سقف لقيمة `take` ، و `RawByBookId` يعيد الخزنة كاملة بلا صفحات	طلب واحد بـ `take=1000000` يستهلك ذاكرة الخادم	سقف ثابت (مثلاً 200)، ونقل التصدير إلى بث (Streaming)
14	`SessionValidationMiddleware` يستعلم قاعدة البيانات في **كل طلب**	استعلام إضافي على كل استدعاء API	تخزين الجلسة في `IMemoryCache` لدقيقة أو دقيقتين مع إبطالها عند تسجيل الخروج
15	سجل التدقيق مضبوط على `LogReadRequests=true` و `LogRequestBodies=true` مع احتفاظ 365 يوماً	كل فتح شاشة يكتب عدة صفوف، فينمو جدول AuditLogs أسرع من جدول الحركات نفسه	إيقاف تسجيل القراءات أو تقليل مدة الاحتفاظ بها، والإبقاء على التغييرات وتسجيل الدخول
16	تسجيل الدخول يستخدم `Username.ToLower() == ...`	لا يُستخدم الفهرس، فيُمسح جدول المستخدمين كاملاً عند كل دخول، و `Include(BusinessUsers)` لا حاجة له	الترتيب الأبجدي (Collation) لا يميّز أصلاً بين الحروف الكبيرة والصغيرة، فتكفي المقارنة المباشرة
17	حذف الخزنة يحمّل كل حركاتها ومرفقاتها إلى الذاكرة ويحوّلها إلى JSON في صف واحد بسلة المهملات	خزنة كبيرة تسبب ارتفاعاً حاداً في الذاكرة أو انتهاء مهلة الطلب	حذف منطقي (عمود `IsDeleted`) بدل نسخ البيانات
18	`CheckWalletAsync` يقرأ جدول BusinessUsers مرتين، ويُستدعى مرتين في النسخ	استعلامات زائدة	قراءة الدور و `BookIds` في استعلام واحد
19	جدول Sessions لا يُنظَّف من الجلسات المنتهية	ينمو بلا حد	تنظيف دوري ضمن `AuditRetentionService`

رابعاً: ثغرات وتقوية أمنية متوسطة

#	المشكلة	الحل
20	**ملفات المرفقات متاحة بلا مصادقة** عبر `/API/Files/{name}` في `Program.cs:190` . إيصالات مالية يمكن فتحها لأي شخص يحصل على الرابط. والاسم لا يمر عبر `Path.GetFileName` ، ونوع المحتوى يُبنى بشكل بدائي `"image/" + ext`	اشتراط المصادقة وفحص أن الملف يتبع خزنة للمستخدم صلاحية عليها، وتنظيف الاسم
21	**رفع الملفات بلا قائمة امتدادات مسموحة ولا حد لحجم الملف**، والحد العام لجسم الطلب 200MB مفعّل على **كل** المسارات (`Program.cs:177-181`)	قائمة بيضاء (jpg وpng وpdf)، وحد 10MB للمرفق، وحد 200MB لمسار النسخ الاحتياطي وحده عبر `[RequestSizeLimit]`
22	رسائل `ex.ToString()` تُرسل للعميل في أكثر من 30 موضعاً. فلتر `ApiErrorSanitizerFilter` يحجب أغلبها، لكنه حل ترقيعي	`UseExceptionHandler` مركزي مع ProblemDetails وتسجيل الخطأ عبر `ILogger`
23	حماية محاولات الدخول مفتاحها اسم المستخدم مع IP، وهي في الذاكرة. خلف Reverse Proxy أو Docker يصبح IP كل الطلبات واحداً، فيستطيع أي شخص قفل حساب أي مستخدم 15 دقيقة	`UseForwardedHeaders` مع الوكيل الموثوق، و `AddRateLimiter` المدمج في .NET 8
24	إعدادات JWT: مدة رمز الدخول 7 ساعات (طويلة)، و `ValidateIssuer/Audience=false` ، و `RequireHttpsMetadata=false` ، والمفتاح يُقرأ بـ `Encoding.ASCII`	تقصير المدة إلى 15-60 دقيقة، وتفعيل Issuer/Audience، وقراءة المفتاح بـ UTF8
25	CORS في الإنتاج يسمح بـ `localhost:3000` و `127.0.0.1:5500` مع `AllowCredentials`	نقلها إلى `appsettings.Development.json`
26	أي شخص يستطيع إنشاء حساب عبر `register` المفتوح، وهذا يضاعف خطورة البند 1	إن كان النظام داخلياً: التسجيل بالدعوة فقط

خامساً: جودة الكود وقابلية الصيانة

27. **آثار فك التجميع:** `if (1 == 0) {}` و `goto IL_xxxx` و `IL_0777: throw null;` و `[CompilerGenerated] internal` و `class Program` وأسماء مثل `ex2` و `sortDirection2` . الكود يعمل لكنه صعب القراءة والتعديل. **الحل:** تنظيف تدريجي ملفاً بملف مع كل إصلاح، لا إعادة كتابة شاملة.
28. **كبت تحذيرات المترجم** في `cashbook.csproj` : `NoWarn` يُخفي CS8602 وCS8600 وCS8604 وغيرها، وهي بالضبط التحذيرات التي كانت ستكشف أخطاء البندين 9 و10. يُنصح بإعادتها تدريجياً.
29. **تكرار ضخم:** نفس كتلة «استخرج userId ثم افحص الدور ثم ابن `_response` » منسوخة في أكثر من 60 دالة، وبثلاث طرق مختلفة لقراءة الـ claim. هذا التكرار سبب مباشر لثغرتي البندين 1 و2. **الحل:** Authorization Handler أو Filter موحّد مثل `[RequireWalletRole(...)]` ، ودالة `User.GetUserId()` واحدة.
30. **لا توجد طبقة خدمات:** المتحكمات تستخدم `ApplicationDbContext` مباشرة إلى جانب المستودعات، ومنطق الصلاحيات موزّع على الطبقتين.

31. **لا توجد اختبارات إطلاقاً.** لنظام مالي يُنصح على الأقل باختبارات تكامل للصلاحيات (من يستطيع ماذا) وللأرصدة.
32. **خلط** `DateTime.Now` **و** `DateTime.UtcNow` ، مثلاً في `UpdateSettingAsync` وترقية المدير في Program.cs، مقابل UtcNow في الحركات.
33. `Console.WriteLine` بدل `ILogger` في عدة مواضع، فلا تظهر الأخطاء في سجلات الإنتاج.
34. `Forbid()` يعيد 403 بجسم فارغ، فتضيع الرسالة العربية التي جُهّزت قبله، والسلوك غير موحّد بين المتحكمات.
35. **مسار الرفع** يُبنى بـ `Directory.GetCurrentDirectory()` في المستودعات و `ContentRootPath` في Program.cs، وقد يختلفان حسب طريقة التشغيل. يُنصح بخدمة تخزين ملفات واحدة.
36. **ملاحظات صغيرة:** ترتيب الخزائن بالرصيد يعتمد على رصيد الدينار فقط. لا يوجد مسار فحص صحة (Health Check) لـ Docker. لا يوجد ضغط للاستجابات.

نقاط قوة يجب الحفاظ عليها

- الاستعلامات النصية في الحركات تستخدم Parameters بشكل صحيح، فلا يوجد SQL Injection.
- سجل التدقيق مبني جيداً: طابور غير متزامن بسعة محددة، وكتابة على دفعات، وإخفاء للبيانات الحساسة.
- كلمات المرور محفوظة بـ BCrypt، والمفتاح السري خارج appsettings، وهناك فحص لطوله عند الإقلاع.
- إصلاحات العزل الأخيرة على عمليات الكتابة صحيحة، وتعليقاتها توضح السبب جيداً.
- الفهارس على Transactions وAuditLogs موجودة ومدروسة في أغلبها.

اقتراح ترتيب الإصلاح

المرحلة	البنود	الجهد التقريبي
1. أمني عاجل	1، 2، 3، 20	يوم إلى يومين
2. سلامة البيانات	4، 5، 6، 7، 8، 9، 10، 11	يوم إلى يومين
3. الأداء	12، 13، 14، 15، 16	يومان (يشمل Migration للفهرس)
4. تقوية	19، 17، 21 إلى 26	يومان
5. جودة مستمرة	27 إلى 36	تدريجي مع كل تعديل

يُقترح البدء بالمرحلة الأولى، لأن البندين 1 و2 قابلان للاستغلال الآن من أي مستخدم مسجّل.

أُعدّ هذا التقرير بالقراءة المباشرة للكود المصدري دون تشغيل اختبارات اختراق. أرقام الأسطر تشير إلى حالة الملفات وقت الفحص.